Lecture 18 — Linear Programming (Part 2)

COMPSCI 308 — Design and Analysis of Algorithms

Xing Shi Cai https://newptcai.gitlab.io

Duke Kunshan University, Kunshan, China

Jan-Mar 2026

Summary

ITA 29.2 Formulating Problems as Linear Programs

- Single-Pair Shortest-Path
- Maximum-Flow
- Minimum-Cost Flow
- Multicommodity Flow
- Maximum Bipartite Matching

Assignments[1]

Practice makes perfect!

Introduction to Algorithms (ITA) 📖 Required Readings:

- Section 29.2.

✏️ Required Exercises:

- Exercises 29.2 — 1, 2, 4–6.

[1] 💣 Assignments will not be collected; however, quiz problems will be selected from them. (This includes both Readings and Exercises.)

ITA 29.2 Formulating Problems as Linear Programs

ITA 29.2 Formulating Problems as Linear Programs

Single-Pair Shortest-Path

Example — Single-Pair Shortest-Path

Suppose that we want to find the weight of the shortest path between s and t using Linear Programming (LP).

Bellman-Ford.

We may be tempted to write —

distance estimate from s to t.

$$
\begin{aligned}
\text{minimize} \quad & d_t \\
\text{subject to} \quad & d_r \le d_s + 2 \\
& d_t \le d_s + 4 \\
& d_t \le d_r + 1 \\
& d_s = 0, d_r \ge 0, d_t \ge 0
\end{aligned}
$$

Distance from s to s is 0.

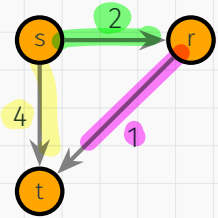


Figure 1: A weighted graph

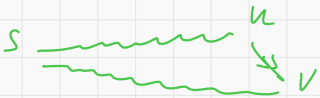
What is the obviously optimal solution?

Triangle ineq: $d_v \le d_u + w(u,v)$

Optimal Solution:

$d_t = d_r = d_s = 0.$

Wrong.



Example — Single-Pair Shortest-Path

Suppose that we want to find the weight of the shortest path between s and t using LP.

What if we change to —

$$
\begin{aligned}
\text{maximize} \quad & d_t \\
\text{subject to} \quad & d_r \le d_s + 2 \quad = 2 \\
& d_t \le d_s + 4 \quad = 4 \\
& d_t \le d_r + 1 \\
& d_s = 0, d_r \ge 0, d_t \ge 0
\end{aligned}
$$

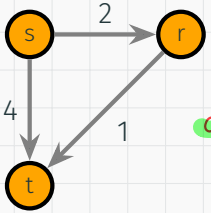


Figure 1: A weighted graph

Optimal objective value

$d_t = 3$

$d_r = 2$

$d_s = 0.$

What is the obviously optimal solution?

$d_r \le 2$

$d_t \le 4$

$d_t \le d_r + 1$

$\Rightarrow$ $d_t \le 4$

$d_t \le 2 + 1 = 3$

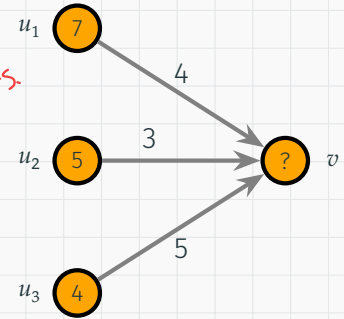
$\Rightarrow d_t \le 3$

The Single-Pair Shortest-Path Problem

Computing the weight of the shortest path from s to t is equivalent to

Triangle Inequalities.

$$\begin{aligned} \text{maximize} \quad & d_t \\ \text{subject to} \quad & d_v \le d_u + w(u,v) \quad \text{for each } (u,v) \in E, \\ & d_s = 0 \end{aligned}$$

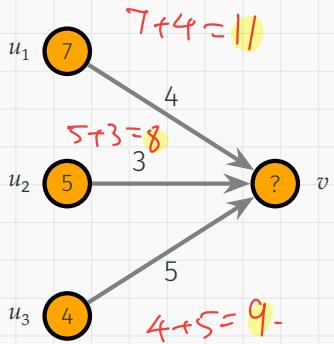


Note that the variables do not need nonnegative constraints, since this is not asking for the standard form of an LP problem.

The Single-Pair Shortest-Path Problem

Computing the weight of the shortest path from s to t is equivalent to

$$\begin{aligned} \text{maximize} \quad & d_t \\ \text{subject to} \quad & d_v \le d_u + w(u,v) \quad \text{for each } (u,v) \in E, \\ & d_s = 0 \end{aligned}$$



The Bellman-Ford algorithm sets

$$d_v = \min\{d_u + w(u,v) \mid (u,v) \in E\}$$

Thus d_v is the *maximum* value that satisfies

$$d_v \le d_u + w(u,v) \quad \text{for each } (u,v) \in E$$

\every edge

$\to v$

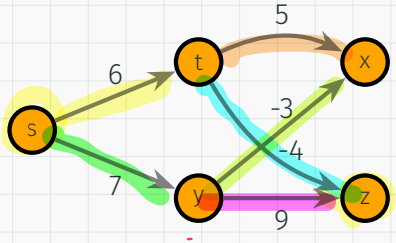
$d_v \le 11$
≤ 9
$\le 9.$

Thus, the maximization makes sense.

Exercise

The optimal objective value should be $6-4=2$.

Formulate the problem of finding the distance from s to z as an LP problem for the graph shown in the right.



maximize d_z

subject to

$d_t \le d_s + 6$

$d_y \le d_s + 7$

$d_x \le d_t + 5$

$d_s = 0.$

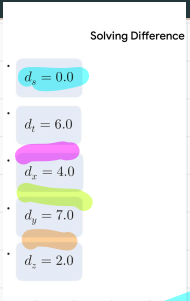
$d_x \le d_y - 3.$

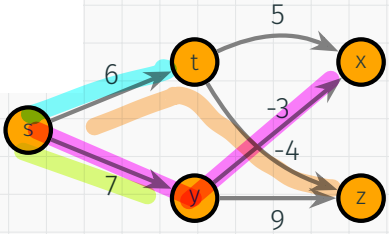
$d_z \le d_y + 9$

$d_z \le d_t - 4.$

Exercise

Formulate the problem of finding the distance from s to z as an LP problem for the graph shown in the right.





maximize d_z

subject to

$$d_t \leq d_s + 6 \qquad d_z \leq d_t - 4$$
$$d_y \leq d_s + 7 \qquad d_z \leq d_y + 9$$
$$d_x \leq d_t + 5 \qquad d_x \leq d_y - 3$$
$$d_s = 0$$

Exercise 29.2-2

Given a graph G, write an LP for the single-source shortest-paths problem. The solution should have the property that d_v is the shortest-path weight from the source vertex s to v for each vertex $v \in V$.

When there is unreachable vertex this won't work.

Find the distance to all vertices.

$$\begin{aligned} \text{maximize} \quad & d_t \rightarrow \sum_{v \in V} d_v \\ \text{subject to} \quad & d_v \le d_u + w(u,v) \quad \text{for each } (u,v) \in E, \\ & d_s = 0 \end{aligned}$$

Yes!

The Bellman-Ford algorithm sets

$$d_v = \min\{d_u + w(u,v) \mid (u,v) \in E\}$$

Exercise 29.2-2

Given a graph G, write an LP for the single-source shortest-paths problem. The solution should have the property that d_v is the shortest-path weight from the source vertex s to v for each vertex $v \in V$.

Solution —

$$\begin{aligned} \text{maximize} \quad & \sum_{v \in V} d_v \\ \text{subject to} \quad & d_v \le d_u + w(u,v) \quad \text{for each } (u,v) \in E, \\ & d_s = 0 \end{aligned}$$

The constraints ensure each d_v is at most the length of any $s \to v$ path.

So maximizing the sum makes every d_v as large as possible until they reach the distance from s.

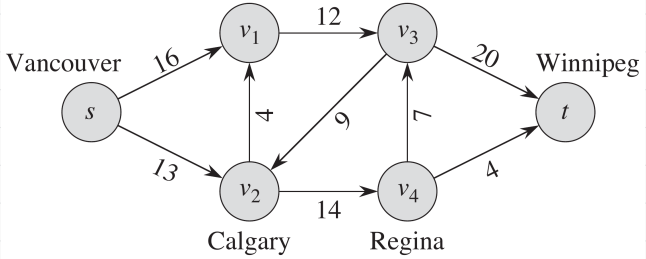
ITA 29.2 Formulating Problems as Linear Programs

Maximum-Flow

Maximum-Flow Problem

Assume that on the 🇨🇦 highway network —

- there is a limit on 🚚 between every two cities,
- the 🚚 coming into a city must go out of it.

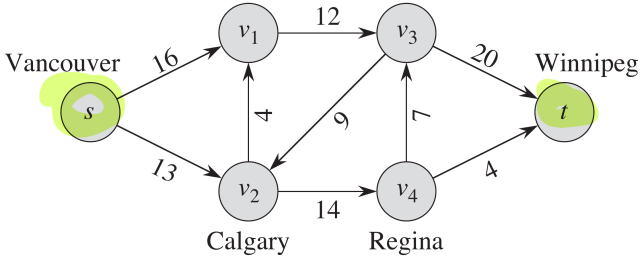


Maximum-Flow Problem

Assume that on the 🇨🇦 highway network —

? Answer: 14 ?

- there is a limit on 🚚 between every two cities,
- the 🚚 coming into a city must go out of it.



What is the maximum 🚚 possible from Vancouver to Winnipeg?

This is called a maximum-flow problem.

Maximum-Flow Problem Formulation

We can find the maximum flow on the network on the right by —

flow out of the source

flow into the source.

$\Rightarrow$ Net out flow

$$
\begin{aligned}
\text{maximize} \quad & f_{st} + f_{sr} - f_{ts} \\
\text{subject to} \quad & f_{sr} \le 2 \\
& f_{st} \le 3 \\
& f_{rt} \le 1 \\
& f_{ts} \le 9 \\
& f_{sr} - f_{rt} = 0, \\
& f_{st}, f_{sr}, f_{rt}, f_{ts} \ge 0
\end{aligned}
$$

constraints due to capacity.

Preserve of flow at intermidate vertices.

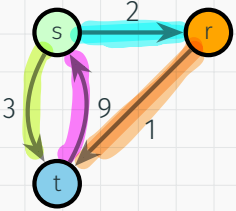


Figure 2: A network with flow constraints

Maximum-Flow Problem Formulation

Finding the maximum flow from a source s to a sink t can be modelled as —

— flow out of s

$$
\begin{aligned}
\text{maximize} \quad & \sum_{v:(s,v)\in E} f_{sv} - \sum_{v:(v,s)\in E} f_{vs} && \text{— flow in to s.} \\
\text{subject to} \quad & f_{uv} \le c(u,v) && \text{for each } (u,v) \in E, \\
& 0 = \sum_{v:(v,u)\in E} f_{vu} - \sum_{v:(u,v)\in E} f_{uv} && \text{for each } u \in V - \{s,t\}, \\
& f_{uv} \ge 0 && \text{for each } (u,v) \in E.
\end{aligned}
$$

Respect capacity

Preserve the flow

Here —

- f_{uv} is the flow from u to v,
- $c(u,v)$ is the capacity from u to v.

Exercise

Please write down the LP formulation for the following maximum-flow problem.

Or: $f_{st} + f_{yt}$

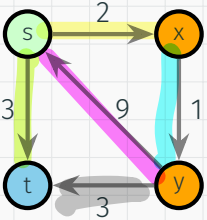


Figure 3: A network with flow constraints

Maximize:

$$f_{st} + f_{sx} - f_{ys}$$

Constraints: $f_{st} \le 3$, $f_{sx} \le 2$.

$f_{xy} \le 1$, $f_{ys} \le 4$

$f_{yt} \le 3$

$f_{sx} - f_{xy} = 0$, $f_{xy} - f_{sy} - f_{yt} = 0$.

All variables ≥ 0.

Exercise

🍰 Please write down the LP formulation for the following maximum-flow problem.

maximize $f_{st} + f_{s,x} - f_{y,s}$

subject to

$$
\begin{aligned}
f_{st} &\le 3 & f_{y,t} &\le 3 \\
f_{sx} &\le 2 & f_{y,s} &\le 9 \\
f_{xy} &\le 1 & f_{s,x} - f_{x,y} &= 0 \\
f_{xy} - f_{y,t} - f_{y,s} &= 0 \\
f_{st}, f_{s,x}, f_{x,y}, f_{y,t}, f_{y,s} &\ge 0
\end{aligned}
$$

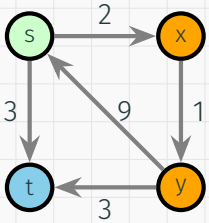


Figure 3: A network with flow constraints

ITA 29.2 Formulating Problems as Linear Programs

Minimum-Cost Flow

Minimum-Cost-Flow Problem

In addition to a capacity $c(u,v)$ for each edge (u,v), suppose we are given a real-valued cost $a(u,v)$.

Sending f_{uv} units of flow over edge (u,v) incurs a cost of $a(u,v) \cdot f_{uv}$.

Given a flow demand d, we want to send d units of flow from s to t while minimizing the total cost $\sum_{(u,v)\in E} a(u,v) \cdot f_{uv}$.

This is called the minimum-cost-flow problem.

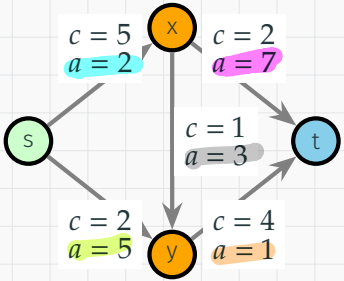
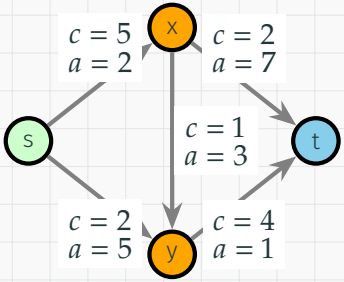


Figure 4: A network with costs and capacities

Exercise

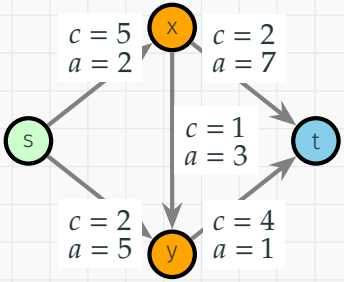
🍰 Please write down the LP formulation for the following minimum-cost-flow problem.



Exercise

🍰 Please write down the LP formulation for the following minimum-cost-flow problem.

$$
\begin{aligned}
\text{minimize} \quad & 2f_{sx} + 5f_{sy} + 3f_{xy} + 7f_{xt} + f_{yt} \\
\text{subject to} \quad & f_{sx} \le 5 \qquad f_{xt} \le 2 \\
& f_{sy} \le 2 \qquad f_{yt} \le 4 \\
& f_{xy} \le 1 \qquad f_{sx} - f_{xy} - f_{xt} = 0 \\
& f_{sy} + f_{xy} - f_{yt} = 0 \qquad f_{sx} + f_{sy} = d \\
& f_{sx}, f_{sy}, f_{xy}, f_{xt}, f_{yt} \ge 0
\end{aligned}
$$



Minimum-Cost-Flow Problem Formulation

Finding a minimum-cost flow of value d can be modelled as —

$$
\begin{aligned}
\text{minimize} \quad & \sum_{(u,v)\in E} a(u,v)\cdot f_{uv} && \text{— total cost} \\
\text{subject to} \quad & 0 \le f_{uv} \le c(u,v) && \text{for each } (u,v)\in E, \\
& 0 = \sum_{v:(v,u)\in E} f_{vu} - \sum_{v:(u,v)\in E} f_{uv} && \text{for each } u \in V - \{s,t\}, \\
& d = \sum_{v:(s,v)\in E} f_{sv} - \sum_{v:(v,s)\in E} f_{vs}
\end{aligned}
$$

The net flow out the source of d.

If d > max flow, then is is infeasible.

ITA 29.2 Formulating Problems as Linear Programs

Multicommodity Flow

Multicommodity Flow Problem

A single shipping network can carry multiple commodities, and each commodity i has a source s_i, a sink t_i, and a demand d_i.

The flow of commodity i on edge (u, v) is f^i_{uv}, and the aggregate flow is $f_{uv} = \sum_i f^i_{uv}$. The aggregate flow must respect the edge capacities.

Example with two commodities:

- Vancouver → Saskatoon
- Edmonton → Regina

This is called the multicommodity-flow problem.

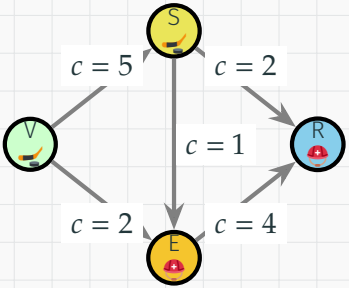


Figure 5: A shared network for two commodities

Multicommodity-Flow Problem Formulation

Suppose that we have k commodities. Then the multicommodity-flow problem can be modelled as —

No need to minize

$$
\begin{aligned}
\text{minimize} \quad & 0 \\
\text{subject to} \quad & \sum_{i=1}^{k} f_{uv}^{i} \leq c(u,v) && \text{for each } (u,v) \in E, \\
& 0 = \sum_{v:(v,u)\in E} f_{vu}^{i} - \sum_{v:(u,v)\in E} f_{uv}^{i} && \text{for each } i \in \{1,2,\dots,k\},\ u \in V - \{s_i, t_i\}, \\
& d_i = \sum_{v:(s_i,v)\in E} f_{s_i v}^{i} - \sum_{v:(v,s_i)\in E} f_{v s_i}^{i} && \text{for each } i \in \{1,2,\dots,k\}, \\
& f_{uv}^{i} \geq 0 && \text{for each } i \in \{1,2,\dots,k\},\ (u,v) \in E
\end{aligned}
$$

The demand for commodity i.

ITA 29.2 Formulating Problems as Linear Programs

Maximum Bipartite Matching

Exercise 29.2-5

In the graph to the right, suppose each edge means an employer is willing to hire that employee.

We want to hire as many employees as possible, while ensuring each employer hires at most one person and each employee accepts at most one job.

Bipartite graph.

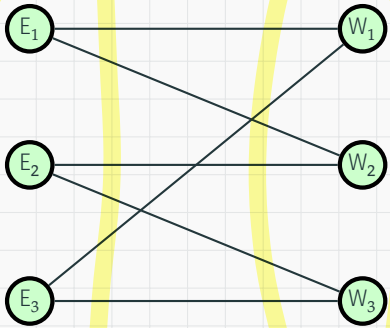


Figure 6: Employers and employees

Exercise 29.2-5

In the graph to the right, suppose each edge means an employer is willing to hire that employee.

We want to hire as many employees as possible, while ensuring each employer hires at most one person and each employee accepts at most one job.

Given a bipartite graph $G = (V, E)$ with partitions L and R, the goal is to find a maximum-size set of edges so that no vertex is used more than once.

This is the maximum-bipartite-matching problem.

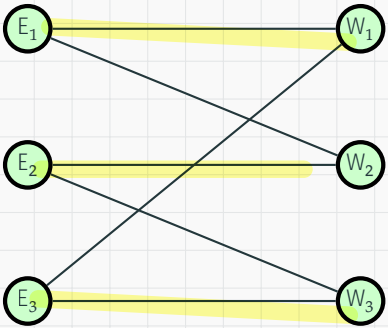


Figure 6: Employers and employees

Maximum Bipartite Matching as Max Flow

☆: Optimal solution does not mean integer flow everywhere

We can solve this problem as a max flow problem.

Add a source s and sink t.

Connect s to every employer with capacity 1, and connect every employee to t with capacity 1.

Direct each employer-to-employee edge from left to right with capacity 1.

Any integer max flow selects a set of nonconflicting hires, and its value equals the matching size.

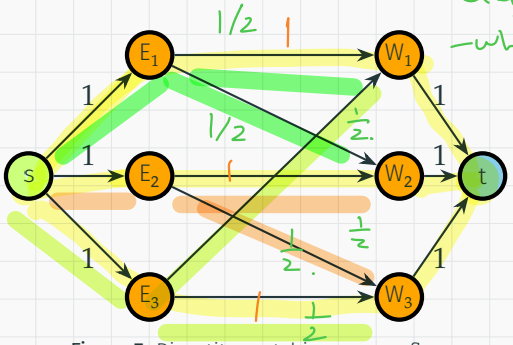


Figure 7: Bipartite matching as max flow

Maximum Bipartite Matching as Max-Flow LP

Using this flow network, we can model the maximum-bipartite-matching problem as —

This LP always has an optimal integer solution.

$$
\begin{aligned}
\text{maximize} \quad & \sum_{v:(s,v)\in E'} f_{sv} \\
\text{subject to} \quad & 0 \le f_{uv} \le c(u,v) && \text{for each } (u,v) \in E', \\
& 0 = \sum_{v:(v,u)\in E'} f_{vu} - \sum_{v:(u,v)\in E'} f_{uv} && \text{for each } u \in V' - \{s,t\}
\end{aligned}
$$

- $G' = (V', E')$ is the flow network built from the bipartite graph.
- f_{uv} is the flow on edge (u,v), and $c(u,v) = 1$.
- The max-flow value equals the matching size.

Variations of Max-Flow Problem

The textbook includes the following problems that can be modelled with LP —

- Maximum Flow
- Minimum-Cost Flow give fixed flow
- Multi-commodity Flow give fixed flow each commodity
- Maximum Bipartite Matching

🤔 Can you think some other problems which are variations of the max-flow problem?

- If $a(u,v)$ is not cost, but profit.
  we want maximize profit for deliver a flow of d.
- Put a lower bound of flow on each edge.

What are your main takeaways today? Any questions?